Azure CLI Permissions in Docker

https://gemini.google.com/app/1cabe9476bc417da

User prompt: how to provide az command permission to a local docker container?

Response: To grant an Azure CLI (`az`) command permission or authenticate a local Docker container so it can interact with Azure, you generally choose one of three approaches depending on your use case: **Mounting host credentials**, **passing environment variables (Service Principal)**, or **logging in interactively inside the container**.

Method 1: Share Your Host Machine's Azure Credentials (Best for Local Development)

If you are already logged into the Azure CLI on your host machine, you can pass your local configuration/credentials folder into the container using a volume mount (`-v`).

- **On Linux / macOS:**

```
docker run -it --rm \
  -v ~/.azure:/root/.azure \
  mcr.microsoft.com/azure-cli:latest \
  az account show
```

- **On Windows (PowerShell):** *(Note: Windows credential encryption sometimes causes issues when read directly by a Linux container. If you get permission or token errors, see the tip below.)*

```
docker run -it --rm `
  -v "$HOME/.azure:/root/.azure" `
  mcr.microsoft.com/azure-cli:latest `
  az account show
```

  💡 **Tip for Windows Users:** If your host is Windows, it's often safer to create a dedicated unencrypted configuration directory for Docker to avoid token decryption issues:

  1. Run a container to log in via device code and save tokens to a distinct folder:

```
docker run -it --rm -v "$HOME/.azure-for-docker:/root/.azure" mcr.microsoft.com/azure-cli az logir
```

  2. Mount that specific folder moving forward:

```
docker run -it --rm -v "$HOME/.azure-for-docker:/root/.azure" mcr.microsoft.com/azure-cli az accou
```

Method 2: Use a Service Principal (Best for Automation & Scripts)

If the container needs to run independently without tying into a personal user account, pass Azure Service Principal credentials as environment variables.

1. Create a service principal on your host machine if you haven't already:

```
az ad sp create-for-rbac --name "my-docker-container-sp" --role Contributor --scopes /subscriptions/<y
```

2. Pass the resulting `appId` (Client ID), `password` (Client Secret), and `tenant` as environment variables when running your container, then log in non-interactively:

```
docker run -it --rm \
  -e AZURE_CLIENT_ID="<appId>" \
  -e AZURE_CLIENT_SECRET="<password>" \
  -e AZURE_TENANT_ID="<tenant>" \
  mcr.microsoft.com/azure-cli:latest \
  sh -c "az login --service-principal -u \$AZURE_CLIENT_ID -p \$AZURE_CLIENT_SECRET --tenant \$AZURE_T
```

Method 3: Interactive Login Inside the Container (Quickest Test)

If you just want to pop into an ephemeral container and log in manually on the fly:

1. Start the container interactively:

```
docker run -it --rm mcr.microsoft.com/azure-cli:latest
```

2. Once inside the container's shell, run:

```
az login
```

   (Follow the prompt to open `[https://microsoft.com/devicelogin](https://microsoft.com/devicelogin)` in your browser and enter the provided code).

3. Run your commands freely (e.g., `az group list`). Note that because the container is ephemeral, these credentials will disappear when the container stops unless you used **Method 1** to mount a volume to persist them.

Sources

1. Using Azure CLI Authentication within Local Containers - endjin

---